

GRUPO ELEKTRA · PLANEACION FINANCIERA

Dashboard Vista Territorio

Documentacion Tecnica

Fecha de generacion	21/07/2026
Version del documento	3.0
Componentes cubiertos	build_data.py · app.py (Streamlit)
Autor / Mantenimiento	Karlo De Jesus Juarez Ramirez

Este documento describe la arquitectura del Dashboard Vista Territorio: como se generan los datos, que hace cada script y como estan organizados los archivos. Dirigido a quien da mantenimiento al pipeline (build_data.py / app.py).

Tabla de Contenidos

1. Overview del sistema
2. Script build_data.py
3. Script app.py
4. Rendimiento y carga diferida
5. Flujo de actualizacion semanal
6. Configuracion minima
7. Estructura de carpetas

1. Overview del Sistema

El Dashboard Vista Territorio compara cuatro medidas (Real 2025, Real 2026, Plan 2026 y Nvo Plan 2026) a lo largo de division, territorio, zona, region, PDC y cuenta contable. Trabaja en dos etapas encadenadas:

Componente	Funcion	Duracion tipica
build_data.py	Lee catalogos y parquets fuente, calcula agregados y los guarda en aggs/.	10-18 min (--rebuild) 5-10 min (--aggs-only)
app.py	Interfaz Streamlit: lee los parquets de aggs/ y dibuja las pestanas.	Tiempo real

Como pensarlo

build_data.py es la 'cocina' (procesa los datos pesados una sola vez); app.py es el 'comedor' (sirve esos datos ya listos). Por eso no hay que reprocesar todo cada vez que alguien abre el dashboard.

2. Script build_data.py

Fuentes de entrada

- Parquets semanales: r25_semNN.parquet, r26_semNN.parquet, p26_semNN.parquet (extraidos de SQL Server via extraer_sql.py)
- nvo_plan_2026.parquet: plan alternativo, calendario de 53 semanas
- Catalogos Excel **dentro de la carpeta del dashboard** (no en la raiz de Downloads): 'Catalogo grupo cuentas.xlsx' y 'Catalogo de estructura.xlsx'

Los catalogos viven junto al script, no en Downloads

CAT_CUENTAS y CAT_ESTRUCT apuntan a BASE (la carpeta del propio build_data.py). Si alguien coloca una copia corregida del catalogo en la raiz de Downloads en vez de aqui, el build seguira leyendo la version vieja sin avisar.

Aggs que genera (carpeta aggs/)

Mas de 25 parquets: agregados por dimension (division, territorio, zona, region, pdc, grupo_cuentas, cuentas...), el consolidado intermedio (_consolidado.parquet), analisis especiales (cierres, forecast_ceco, pdc_pospre) y metadatos de control (_meta.json, orden_gpo.json, responsables_gpo.json, riesgos.json).

pdc_pospre.parquet — agg dedicado al drill-down territorial

Cruza Grupo de Cuentas, Cuenta, Division/Territorio/Zona/Region, PosPre y PDC para las 2 vistas de la pestana Detalle Cuenta. cat_PosPre y cat_Denom_PosPre se concatenan en una sola columna (cat_PosPre_Full) antes del agrupado: son casi 1:1, tratarlos como 2 niveles de arbol solo multiplicaba filas sin aportar jerarquia real. Aun asi el agg resulta grande (~17.7M filas) porque Territorio/Zona/Region en el catalogo NO son una jerarquia limpia (un mismo Territorio aparece bajo ~20-25 combinaciones de Zona/Region) y se cruza con PDC (4,115 valores). Ver seccion 4 para como se resuelve esto en app.py.

Como se ejecuta

```
python build_data.py --rebuild      # Reconstruir _consolidado desde los parquet fuente
python build_data.py                # Reusa _consolidado.parquet si sigue vigente
python build_data.py --aggs-only    # Recalcula solo aggs/, sin releer fuentes ni catalogos
python build_data.py --semana 30    # Fuerza una semana de corte especifica
```

Cuando usar --rebuild

Usa --rebuild si cambiaron los catalogos Excel o los parquet fuente (r25/r26/p26_semNN). Para regenerar solo un agg puntual sin tocar el consolidado, --aggs-only es mucho mas rapido.

Riesgos y Oportunidades (comentarios de Planeacion)

build_riesgos() lee 'Riesgos y Oportunidades.docx' (en la carpeta del dashboard) y genera aggs/riesgos.json. Formato del Word: un parrafo 'RIESGOS' abre la seccion de variaciones negativas, 'OPORTUNIDADES' abre la de positivas; cada parrafo debajo es una vineta. Se puede regenerar solo, sin correr el build completo:

```
python -c "import build_data; build_data.build_riesgos()"
```

3. Script app.py

Interfaz Streamlit que lee los parquet de aggs/ y dibuja las pestanas, sin tocar las fuentes originales. Login por usuario/contrasena (usuarios.json).

Pestanas actuales

Resumen, Div / Terr, Detalle Cuenta, Detalle PDC, PDC & Calor, Forecast CECO, Sem / Mes, Cierres, Trimestres, Riesgos & Oport., Comentarios.

La pestana 'Movimiento' se retiro

Se elimino de la navegacion y del codigo el 21/07/2026 (a peticion del usuario). movimiento.parquet se sigue generando en el build por si se necesita para analisis fuera del dashboard, pero ya no se carga ni se muestra en app.py.

4. Rendimiento y Carga Diferida

Dos optimizaciones clave para que la app cargue rapido incluso con aggs grandes (conciliacion 758MB, pdc 193MB, pdc_pospre 331MB):

Carga diferida de aggs pesados

load_all() ya no lee conciliacion.parquet, pdc.parquet ni cierres_det.parquet al arrancar. La clase _AggsDiferidos los carga (con spinner) la primera vez que una pestana los pide, vía __missing__. El arranque paso de leer ~1.1 GB a ~25 MB.

Drill-down perezoso (arbol_perezoso)

La pestana Detalle Cuenta usa `pdcp_pospre.parquet` (17.75M filas): pre-armar el arbol completo en HTML de una sola vez colgaba el navegador. `arbol_perezoso()` calcula cada nivel SOLO cuando el usuario abre esa rama (via `pl.scan_parquet + filter + collect` sobre el subconjunto ya acotado por los padres elegidos). No usa `st.expander` (Streamlit no permite anidarlos mas de 1 nivel); usa `st.button + st.session_state` para el toggle abierto/cerrado, y agrupa las hojas de una rama en un solo `st.markdown` en vez de un widget por hoja (evita saturar el canal websocket con miles de widgets por rerun).

read_xlsx() cacheado

`functools.lru_cache` evita releer el mismo catalogo Excel 5 veces por corrida de build (antes se leia una vez por cada funcion que lo necesitaba).

5. Flujo de Actualizacion Semanal

Paso	Descripcion
1	SQL Server acumula los movimientos de la semana en curso.
2	<code>extraer_sql.py --check</code> compara filas/monto de SQL vs. el parquet local mas reciente.
3	Si hay diferencias, <code>extraer_sql.py</code> descarga <code>r25/r26/p26_semNN.parquet</code> nuevos.
4	<code>python build_data.py --rebuild</code> regenera <code>_consolidado.parquet</code> y todos los aggs.
5	<code>app.py</code> detecta el nuevo <code>_meta.json</code> (mtime) e invalida su cache automaticamente.

6. Configuracion Minima

1. **usuarios.json** con al menos un usuario administrador:

```
{"admin": {"password": "hash_sha256", "rol": "admin"}}
```

2. **Catalogos, dentro de la carpeta del dashboard**

- **Catalogo grupo cuentas.xlsx** — obligatorio
- **Catalogo de estructura.xlsx** — obligatorio
- **Riesgos y Oportunidades.docx** — opcional, habilita comentarios de Planeacion

7. Estructura de Carpetas

```
Dashboard Vista Territorio/
|-- build_data.py          # Genera agregados (aggs/)
|-- app.py                 # Interfaz Streamlit
|-- actualizar.py          # Orquesta la extraccion semanal
```

```
|-- extraer_sql.py          # Consulta SQL Server
|-- usuarios.json          # Credenciales y roles
|-- Catalogo grupo cuentas.xlsx
|-- Catalogo de estructura.xlsx
|-- Riesgos y Oportunidades.docx
|-- aggs/                  # Parquets de salida (se regeneran)
|-- tests/                 # Pruebas automatizadas
|-- _respaldo/             # Copias historicas de scripts y docs
```

Documento generado automaticamente el 21/07/2026 14:56 · Version 3.0 · Mantenido por Karlo De Jesus Juarez Ramirez